Chapter V – Supervised Learning

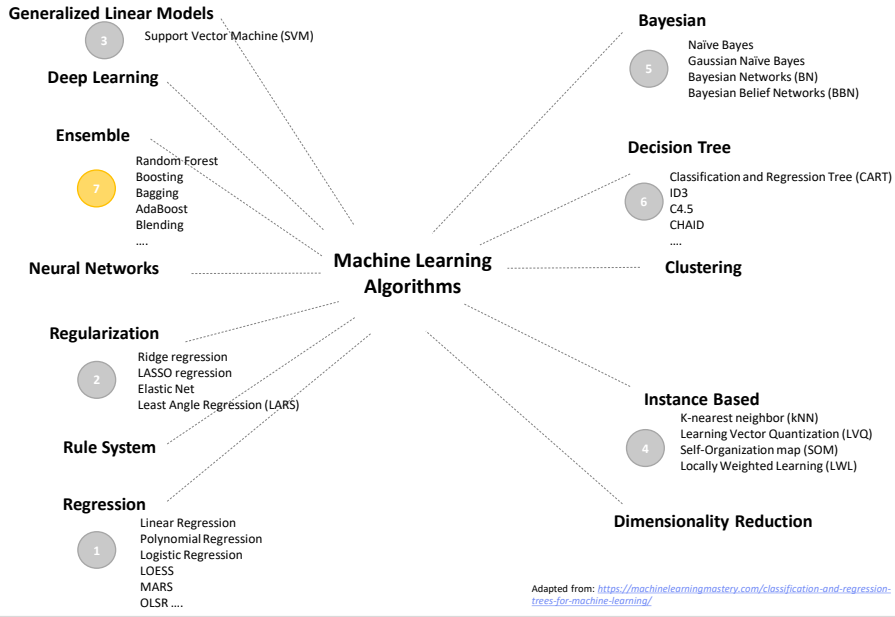


Simão Paredes
sparedes@isec.pt

Ensembles

Classification

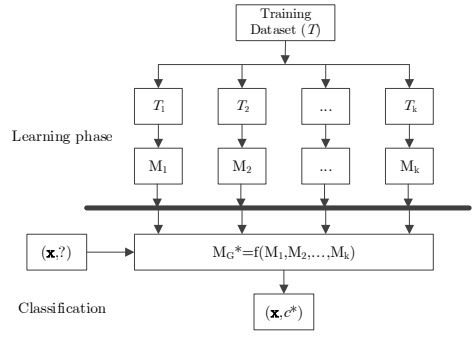
Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Ensemble Learning

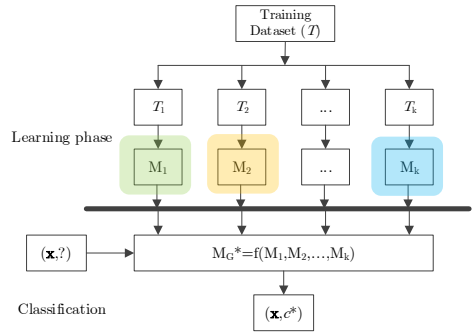
- An ensemble (group) of classifiers is **often** more accurate that any of the single classifiers in the ensemble.



- M_i denotes an individual model
- M_G is the global model
  - It is obtained through the integration of all predictions of M_i

Ensemble Learning

- Important!
  - The individual models should be as independent from one another as possible.



  - This diversity is a key aspect to improve the performance of the ensemble.
  - The integration procedure, i.e. the method to combine predictions of individual classifiers, is essential to define the efficiency of combination.

Ensembles: Voting

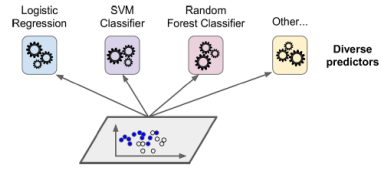
Classification

Ensemble Learning

- Voting
  - It is the simplest integration method, but might be effective
    - The final result is **based on votes of the individual models outputs**.
      - The output of each individual model is considered as a vote for a given class
        - The class with higher number of votes is identified as the final classification.
    - **Weighted voting** is a more elaborate method
      - A given weight is assigned to each individual model.
    - **Dynamic voting** states that the performance of each individual model can change according to the particular characteristics of each instance.
      - The weights of the different models are dynamically adjusted during the combination process.

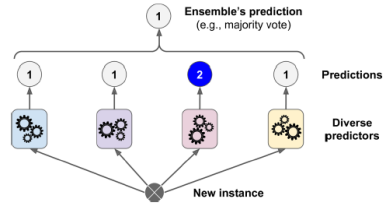
Ensemble Learning

- Voting
  - Considering several individual classifiers:



Diversity of individual classifiers!

  - It might be possible to create a better classifier, through the aggregation of the individual predictions. In the simplest case implements a majority vote scheme (hard voting classifier*)

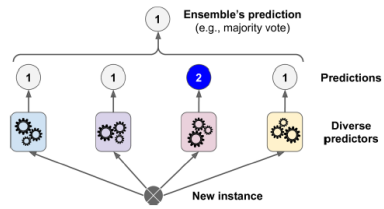


* Instead of this hard identification, if all the classifiers are able to estimate probabilities, the output class would be the class with the highest probability averaged over all individual classifiers (***soft voting***)

Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, ISBN-13: 978-1492032649, O'Reilly 2022

Ensemble Learning

- Voting
  - This simple majority voting scheme can often achieve good performances when compared with individual classifiers

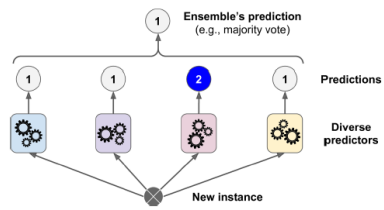


"… even if each classifier is a weak learner (meaning it does only slightly better than random guessing) the ***ensemble can be still be a strong learner*** *(achieving high accuracy), provided there are a sufficient number of weak learners and they are* ***sufficiently diverse****"*

Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 2nd Edition, ISBN-13: 978-1492032649, O'Reilly 2019

Ensemble Learning

- Diversity of Classifiers



  - The performance of ensembles increase when the predictors are as independent from one another as possible.
    - This diversity might be achieved based on the:
      - **selection of different algorithms** to create the base models.
      - **training data** set (e.g. subsets of the original training data, subsets of features, resampling/bootstrapping, …)

Ensemble Learning

- Prediction
  - The majority voting can be stated as:

$$f\colon R^p \rightarrow C$$

$$c_{\mathbf{x}_q} \leftarrow argmax_{c \in C} \sum_{i=1}^{k} \omega_i \delta\left(c, c_{\boldsymbol{M}_i}\right) \qquad \delta(a,b) = \begin{cases} 1 & if \quad a = b \\ 0 & if \quad a \neq b \end{cases}$$

where:

$\mathbf{x}_q$ is the query instance with p features

$M_1, \ldots., M_k$ are the k individual predictors

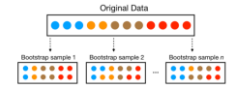
ω_i is the weight factor; $\omega_i = 1$ if all the votes have the same importance.

Ensembles: Bagging

Classification

Bagging

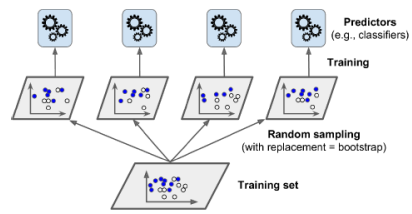
- Bagging / Pasting
  - an ensemble that applies **the same algorithm** (individual models) but it is trained in different random subsets of the training set.



    - Bagging
      - The sampling is performed **with replacement**
        - It allows training instances to be sampled several times for the same predictor.
    - Pasting
      - Sampling is performed **without replacement**
        - It does not allow that training instances can be sampled more than one time for the same predictor.
    - Both approaches allow training instances to be considered several times across multiple predictors

Bagging

- Bagging can be applied both to classification and regression



  - The ensemble makes the prediction for a new instance aggregating the predictions of the individual predictors:
    - Classification is defined like a hard voting classifier (most frequent prediction) [it is a particular case of voting]
    - Regression (typically the average of the individual predictions is carried out)
    - Predictors can be trained in parallel, so the individual predictions can also be made in parallel.
    - Bagging approach scales very well.

Bagging

- Bagging
  - The sampling is based on bootstrapping, which means that a given instance may be considered several times while other instance might not be sampled.
    - The set of no sampled instances is designated as out-of-bag.
      - As expected, these out-of-bag instances differ among the several individual predictors.



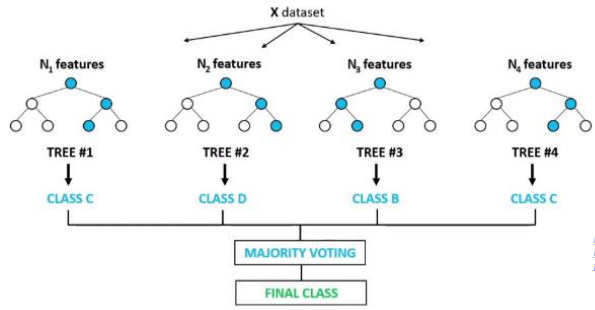
  - The **diversity is obtained through samples** (derivation of different samples) instead of different individual classifiers.

Ensembles: Random Forest

Classification

Random Forest

- Random Forest
  - It is a tree-based supervised learning algorithm based on an ensemble of Decision Trees

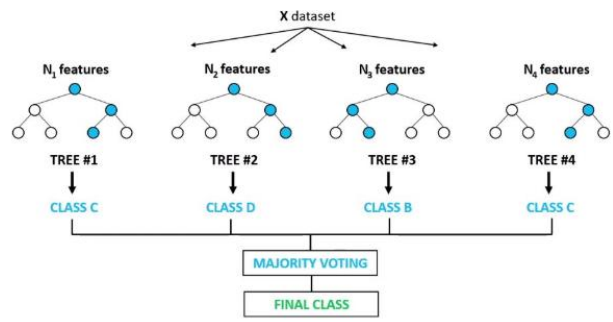


https://www.freecodecamp.org/news/how-to-use-the-tree-based-algorithm-for-machine-learning/

  - Trained based on bagging method:
    - Dozens/Hundreds of individual decision trees (individual predictors) are combined where each decision tree is trained on a different sample of the observations.

Random Forest

- The final prediction of the random forest is obtained:
  - Through the most frequent class (classification) — Classification !!
  - Averaging the predictions of each individual tree — Regression !!



Random Forest

- The required diversity is implemented through two different perspectives:
  - Each tree uses a **random part** of the original data as training set
    - By convention, sampled from the original dataset with replacement
  - Only a **random subset** of input features are sampled for split points at each node in the trees produced.
    - Each tree in Random Forest usually considers $\sqrt{N}$ of the N available features
      - $\sqrt{N}$ are chosen at random
- Thus, Random Forest combines multiple decision trees:
  - Training each one on a slightly different set of the observations
  - Splitting nodes in each tree considering a limited number of the features.

Random Forest

- Random Forest is characterized by:
  - **Random** sampling of training **instances**
    - Training, each tree in a random forest learns from a random sample of the data points.
    - The samples are drawn with replacement (bootstrapping)
      - Some samples will be used multiple times in a single tree.
    - Training each tree on different samples will reduce the variance of the entire forest.
  - **Random** Subsets of **features** for splitting nodes
    - Only a subset ($\sqrt{N}$) of all the features are considered for splitting each node in each decision tree.
    - The individual decision trees tend to overfit to the training data
      - Random forest can mitigate that issue by considering that prediction results from multiple trees.

Random Forest

- **Feature importance**
  - The Random Forest algorithm also allows to identify the most important features in the dataset
    - This is a very important characteristic of Random Forest classifier.
  - The feature importance is measured though the consideration of how many tree nodes apply that feature (on average) to reduce impurity across all trees in the forest.

```
1  from sklearn.datasets import load_iris
2  iris=load_iris()
3
4  from sklearn.ensemble import RandomForestClassifier
5  rnd_clf=RandomForestClassifier(n_estimators=500, n_jobs=-1)
6  rnd_clf.fit(iris['data'],iris['target'])
7
8  for name,score in zip(iris['feature_names'],rnd_clf.feature_importances_):
9      print(name,score)
10
```

```
sepal length (cm) 0.10487678476339472
sepal width (cm) 0.02203605500716085
petal length (cm) 0.4222051240892053
petal width (cm) 0.45088203614023914
```


Random Forest

- Random Forest when compared with Decision Tree:
  - is a more complex classifier
  - is less prone to overfitting situations
  - has higher performance
  - has lower variance (less vulnerable to changes in the training set)
  - does not assure interpretability (black box model)
    - **Important drawback!!**

Random Forest

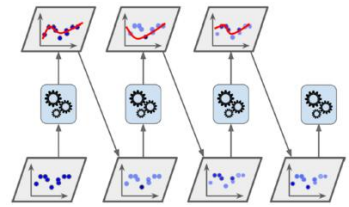
- Random Forest when compared with Decision Tree:
  - black box model (drawback)
    - It cannot be applied when human interpretability of the final model is of primary concern.
    - Decision Trees are intuitive, as their decisions are easy to interpret as a set of rules.
      - Such models are called **white box models** in contrast with black box models (e.g. random forest, Neural networks).
        - Black Box models usually present very high performances, however:
          - Their predictions are hard to explain in simple terms!
          - In some contexts it is an obstacle to their effective application!

Ensembles: Boosting

Classification

Boosting

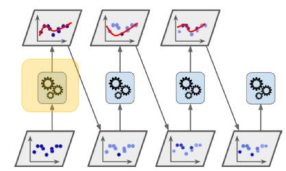
- The approach of boosting is based on the idea of:
  - training predictors **sequentially**, trying to improve the previous prediction.
    - This is a completely different perspective from bagging, as in this situation individual predictors can not be considered in parallel nor independently.



  - AdaBoost
    - It creates a sequence of predictors that try to progressively improve the prediction
      - Throughout its operation, it focuses the instances that were misclassified by the previous classifiers.

Boosting

- AdaBoost
  - Initially all the instances have the same weight:



$$w^{(i)} = \frac{1}{m}$$

being m the number of instances.

  - A first predictor is trained and its weighted error rate r_j is computed on the training set, such as:

$$r_j = \frac{\sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^{m} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

where, $\hat{y}_j^{(i)}$ is the jth predictor's prediction for the ith instance

Boosting

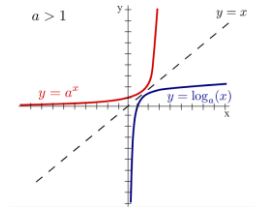
- AdaBoost

$$r_j = \frac{\sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^{m} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

  - $r_j \approx 1$ (high) means a classifier with poor performance (high number of $\hat{y}_j^{(i)} \neq y^{(i)}$)
  - $r_j \approx 0.5$ is a random classifier
  - $r_j \approx 0$ (low) means a classifier with high performance (low number of $\hat{y}_j^{(i)} \neq y^{(i)}$)
  - The predictor weight is computed as:

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

where η is the learning rate hyper parameter (default $\eta = 1$)



Boosting

- AdaBoost

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

  - The **more accurate** the predictor **the higher is the respective α_j**
  - The instances' weights are updated in order to give more importance to the **misclassified instances**:

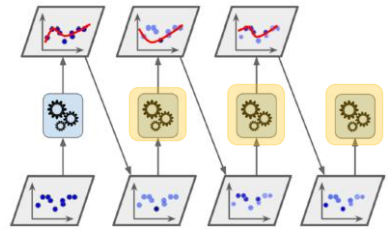
$$w^{(i)} = \begin{cases} w^{(i)} \exp(-\alpha_j) & if \quad \hat{y}_j^{(i)} = y_j^{(i)} \\ w^{(i)} \exp(\alpha_j) & if \quad \hat{y}_j^{(i)} \neq y_j^{(i)} \end{cases} \quad , for = 1,2 \ldots, m$$

After update the weights, they are normalized:

$$w_{new}^{(i)} = \frac{w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

Boosting

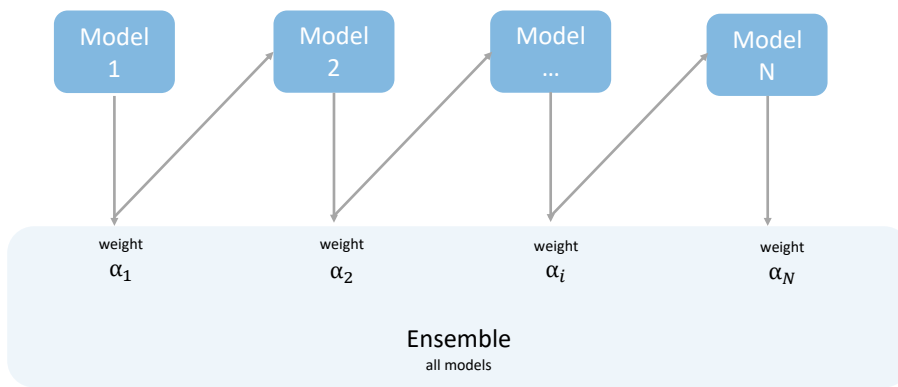
- A new dataset has to be created using the normalized weights, increasing the probability of selecting the wrong instances for training purposes.
- A new predictor is trained using the new weights for the instances
- The whole process is repeated:



the algorithm stops when the number of predictors is reached or when there are no incorrectly classified instances.

Boosting

- AdaBoost



Boosting

- Adaboost Prediction
  - All the N predictors are considered as well as the respective weights α_j (corresponding to the performance of each predictor)
  - The predicted class is the one that receives the majority of the weighted votes:

$$\hat{y}(\mathbf{x}) = argmax_{c \in C} \sum_{\substack{j=1 \\ \hat{y}_j^{(i)}=c}}^{N} \alpha_j$$

  - For a new instance, each predictor produces an output (e.g. 0/1) with the respective weight (α_j).
    - The sum of these α_j for each class (e.g. 0/1) originates the the final output.

Boosting

- Example:
  - Adaboost Operation:

#	x	y
1	1	+1
2	2	+1
3	3	–1
4	4	–1

  - Start with a weak classifier (round 1)
    - Simple Rule: "Predict +1 if $x < \theta$, else -1"
  - Each point has equal Weight
    - $\omega_i = \frac{1}{4} = 0.25$
  - Considering $\theta = 3.5$
    - Predictions: $x = 1 \rightarrow +1$ ✓
      $x = 2 \rightarrow +1$ ✓
      $x = 3 \rightarrow +1$ ✗
      $x = 4 \rightarrow -1$ ✓

Boosting

- Example:
  - Adaboost Operation:
    - error: $\varepsilon_1 = \omega_3 = 0.25$
    - alpha: $\alpha_1 = \frac{1}{2}\ln\left(\frac{1-\varepsilon_1}{\varepsilon_1}\right) = \frac{1}{2}\ln\left(\frac{0.75}{0.25}\right) = \frac{1}{2}\ln(3) \cong 0.55$
    - Update Weights:
      - Instances classified correctly: $\omega_i' = 0.25.e^{-\alpha_1} = 0.25.e^{-0.55} \cong 0.144$
      - **Misclassified instances:** $\omega_3' = 0.25.e^{+\alpha_1} = 0.25.e^{0.55} \cong 0.432$
        - Normalized weights: $\sum_{i=1}^{4}\omega_i' = 0.144 + 0.144 + 0.432 + 0.144 = 0.864$
          - $\omega_1 = \frac{0.144}{0.864} = 0.1667$
          - $\omega_2 = 0.1667$
          - $\omega_3 = 0.5$
          - $\omega_4 = 0.1667$

Boosting

- Example:
  - Adaboost Operation:

#	x	y
1	1	+1
2	2	+1
3	3	–1
4	4	–1

    - Second Classifier (Round 2)
      - Predict +1 if $x \quad < \theta$, else -1
    - Considering $\theta = 1.5$
      - Predictions: $x = 1 \rightarrow +1$ ✓
        $x = 2 \rightarrow -1$ ✗
        $x = 3 \rightarrow -1$ ✓
        $x = 4 \rightarrow -1$ ✓

Boosting

- Example:
  - Adaboost Operation:
    - error from round 1: $\omega^{(2)} = [0.1667, 0.1667, 0.5, 0.1667]$
    - alpha: $\alpha_2 = \frac{1}{2}\ln\left(\frac{1-\varepsilon_2}{\varepsilon_2}\right) = \frac{1}{2}\ln\left(\frac{0.8333}{0.1667}\right) = \frac{1}{2}\ln(5) \cong 0.804$
    - Update Weights:
      - Instances:

$$\omega_1' = 0.1667.e^{-0.804} \cong 0.07453$$
$$\omega_2' = 0.1667.\boldsymbol{e^{+0.804}} \cong 0.3727 \quad (misclassified)$$
$$\omega_3' = 0.5.e^{-0.804} \cong 0.22361$$
$$\omega_4' = 0.1667.e^{-0.804} \cong 0.07453$$

- Normalized weights: $\sum_{i=1}^{4} \omega_i' = 0.753$

$$\omega^{(3)} = [0.1, 0.5, 0.3, 0.1]$$

Boosting

- Example:
  - Adaboost Operation:
    - Compute Scores $\hat{y} = sign\big(S(x)\big)$

$$S(x) = \sum_{t=1}^{T} \alpha_t \, h_t(x)$$

$x = 1$ $S^{(2)}(1) = 0.55(+1) + 0.804(+1) = 1.35 \rightarrow \hat{y} = +1$

$x = 2$ $S^{(2)}(2) = 0.55(+1) + 0.804(-1) = -0.255 \rightarrow \hat{y} = -1\ (wrong)$

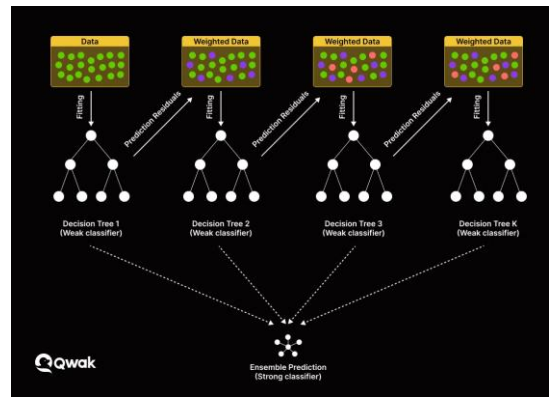
$x = 3$ $S^{(2)}(3) = 0.55(+1) + 0.804(-1) = -0.255 \rightarrow \hat{y} = -1$

$x = 4$ $S^{(2)}(4) = 0.55(-1) + 0.804(-1) = -1.35 \rightarrow \hat{y} = -1$

XGBoost – Extreme Gradient Boosting

XGBoost

- XGBoost (Extreme Gradient Boosting)
  - builds decision trees sequentially with each tree attempting to correct the mistakes made by the previous one
  - Optimized implementation of gradient boosted decision tree:
    - Speed
    - Accuracy
    - Scalability
  - It can be applied to problems of:
    - Classification
    - Regression
  - https://doi.org/10.1145/2939672.29397



https://www.qwak.com/post/xgboost-versus-random-forest

XGBoost

- 1. the algorithm starts with a base learner
  - The first model decision tree is trained on the data.
    - In regression tasks this base model simply predicts the mean of the target variable.
- 2. Calculate the errors
  - After training the first tree the errors between the predicted and actual values are calculated.
- 3. Train the next tree
  - The next tree is trained on the errors of the previous tree, attempting to correct the errors made by the first tree.
- 4. Repeat the process
  - This process continues with each new tree trying to correct the errors of the previous trees until a stopping criterion is met.
- 5. Combine the predictions
  - The final prediction is the sum of the predictions from all the trees.

XGBoost

- The model can be represented as:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i)$$

  - $\hat{y}_i$ is the final prediction for the ith instance
  - K is the number of trees in the ensemble
  - $f_k(x_i)$ represents the prediction of the kth tree for the ith instance

- The global loss function is given by:

$$J(\theta) = \sum_{i}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

  - $l(y_i, \hat{y}_i)$ the difference between the estimated and the actual value of y
  - $\Omega(f_k)$ the regularization term to control the complexity of individual trees

XGBoost

- The model is optimized iteratively:
  - It starts with:

$$\hat{y}_i^{(0)} = 0$$

  - at each step:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

    - $\hat{y}_i^{(t-1)}$ is the prediction from the previous iteration
    - $f_t(x_i)$ is the prediction of the tth tree for the ith instance

  - The regularization term $\Omega(f_t)$ simplify complex trees penalizing the number of leaves as well as the size of the tree.

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} \omega_j^2$$

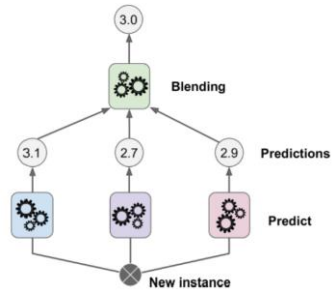
    - T is the number of leaves in the tree; γ regularization parameter that controls the complexity of the tree; λ penalizes the squared weight of the leaves

Ensembles: Stacking

Classification

Stacking

- Stacking
  - This ensemble differs from the simple voting scheme, by introducing a new element that intends to perform a better aggregation of the individual predictions
    - Thus, the aggregation is performed based on a model (blender) that is trained to carry out this aggregation
      - The blender is trained and applied to new predictions, instead of a simple majority class (classification) or averaging the individual predictions (regression)



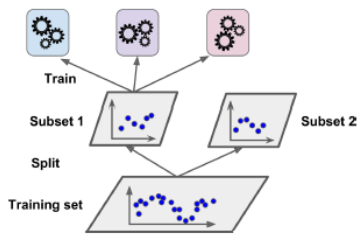
Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, ISBN-13: 978-1492032649, O'Reilly 2022

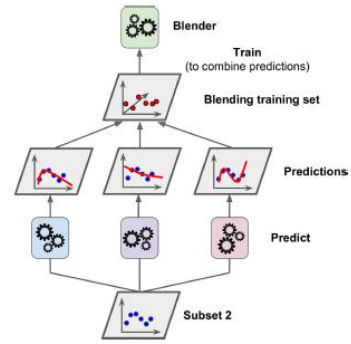
Stacking

- Blender Component
  - A common approach to train the blender is a hold-out set, splitting the original training set into two datasets:

One for training the individual models

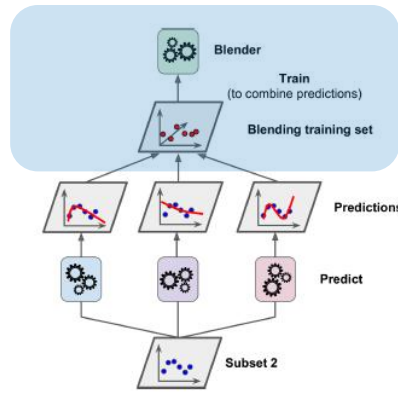
The other applied to train the Blender model





Stacking

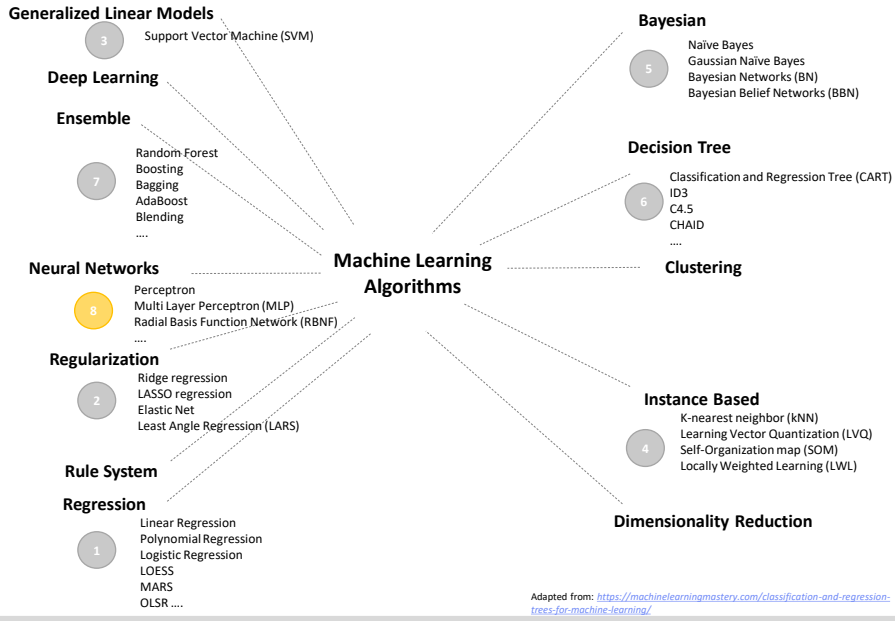
- The original instances are applied to obtain the outputs of individual models
- Then, the training set of the blender uses **the individual predictions as the input features** keeping the original target values.
  - It is possible to create a more complex scheme, creating several layers of predictors (splitting the original training set in more than two different datasets)



Artificial Neural Network

Classification

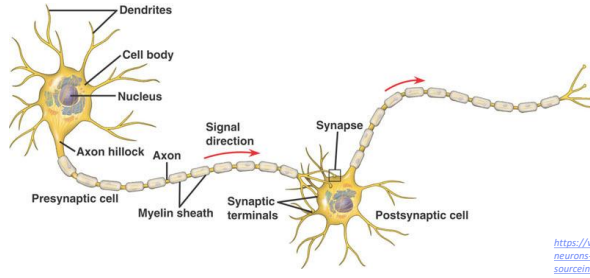
Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Neurons

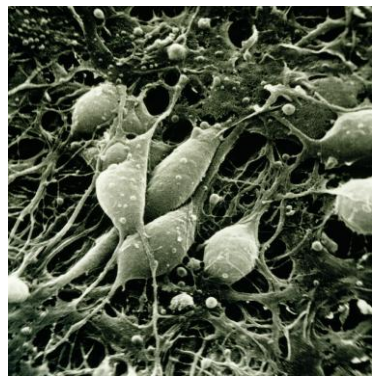
- Biological Neuron
  - produce short electrical impulses (signals) which travel along the axons and make the axon terminals (synapses) release neurotransmitters
  - The synapses are connected to the dendrites of other neurons, creating a neural network
    - When a neuron receives a **sufficient amount of neurotransmitters** within a few milliseconds, it fires its own electrical impulses.



https://www.researchgate.net/figure/1-Two-neurons-having-a-synaptic-connection-sourceinternet_fig1_316553702

Neurons

- Neural Network
  - Individual biological neurons are organized in a vast network of billions
    - Each neuron is connected to thousands of other neurons
    - Highly complex processing can be performed by a network of simple neurons that are organized in consecutive layers



"neurons that fire together wire together."

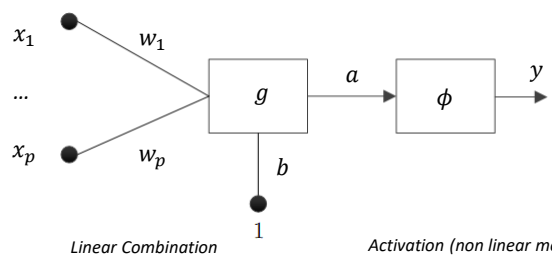
Donald Hebb, Canadian Neuropsychologist, 1949

"Sem Of Neurones From The Human Cerebral Cortex Scanning electron micrograph (SEM) of neurones (nerve cells) from the human cerebral cortex (the outer, heavily folded, grey matter of the brain). Neurones exist in varying sizes & shapes throughout the nervous system, but all have similar basic structure; a large central cell body (large, light grey bodies) containing a nucleus & processes of 2 types: a single axon (a nerve fibre) which is the effector part of the cell that terminates on other neurones (or organs), & one or more dendrites, smaller processes that act as sensory receptors. Similar types of neurones are arranged in layers within the cerebral cortex. Mag: X 3,000 at 8x8 inch, X 890 at 6x6cm size"

https://fineartamerica.com/featured/sem-of-neurones-from-the-human-cerebral-cortex-cnriscience-photo-library.html

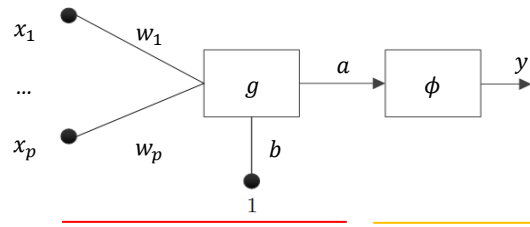
Artificial Neural Networks (ANN)

- ANN are an important class of classifiers, based on biological neural network, that are applied to several real world classification problems.
  - ANN are defined by:
    - The **number of neurons** and by the **way they are connected**
    - **Activation function** $\phi(\cdot)$
  - Perceptron model
    - It is the simplest ANN architecture that is the basic processing unit of a multilayer perceptron neural network.



Perceptron

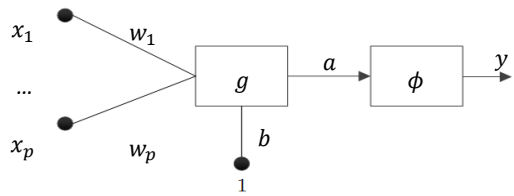
- Model



  - where $\mathbf{x} = [x_1, \ldots, x_p]$ are the inputs,
  - y is the output
  - b is the bias
  - the weight vector is represented as $\mathbf{w} = [w_1, \ldots, w_p]$
    - g is the **combination function** **Linear !**
    - ϕ is the **activation function** **Non-linear !**

Perceptron

- Model



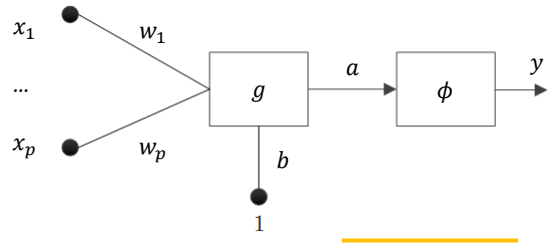
  - Linear Combination
    - Given that $b \in R$; $\mathbf{x}, \mathbf{w} \in R^p$, a is given by

$$a = g(\mathbf{x}, b, \mathbf{w}) = b + \sum_{i=1}^{p} w_i x_i$$

Linear Combination

Perceptron

- Model



  - Activation Function
    - There are several activation functions

$$y = \phi(a) = \phi(b + \sum_{i=1}^{p} w_i x_i)$$

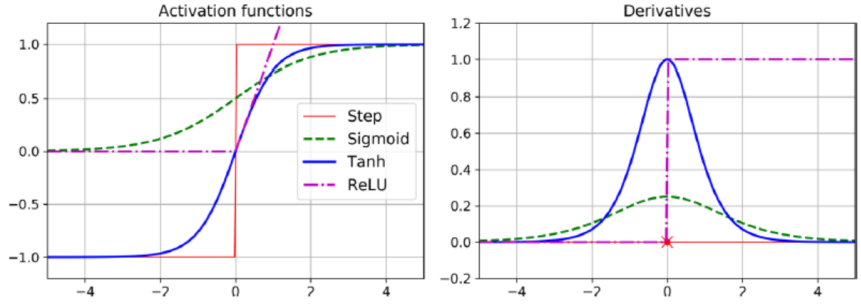
Activation
(non linear model)

Perceptron

- The simplest activation function is the Binary Step Function

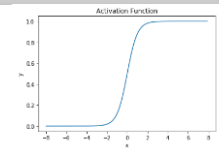
$$y = \phi(a) = \begin{cases} 0 \ if \ \ a < 0 \\ 1 \ if \ \ a \geq 0 \end{cases}$$

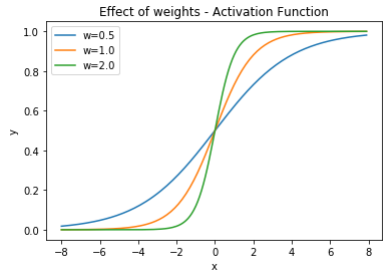
  - There are several linear/non linear activation functions that allow derivatives (gradient descent):



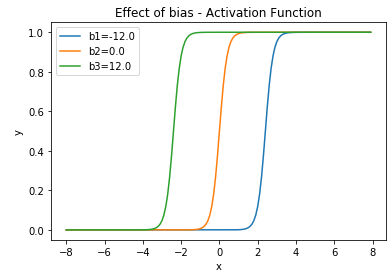
Perceptron

- Activation Function: $y = f(a) = \frac{1}{1+e^{-a}}$





$\mathbf{w} = [w_1, \ldots, w_p]$



b

Weights:

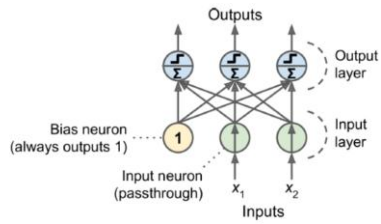
- **change the slope** of the output of the sigmoid activation function (strength of relationship between the input and output variables).
- **do not change** the threshold to activate the neuron

Bias:

- allow to change the threshold (i.e. when the node activates)

Perceptron

- Simple architecture based on the Perceptron model



Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, ISBN-13: 978-1492032649, O'Reilly 2022

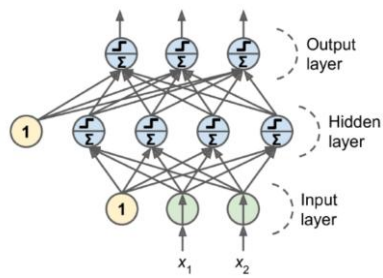
- Output layer
  - composed of a set of neurons connected to **every** neuron (fully connected/dense layer) in the previous layer (input layer)
- Input layer
  - composed of a set of special passthrough (output the respective inputs) neurons (input neurons)
    - the values of the several features are multiplied by the connections' weights
- bias neuron
  - outputs always 1
    - the output is multiplied by the bias value

ANN: Multilayer Perceptron

Classification

Multilayer Perceptron (MLP)

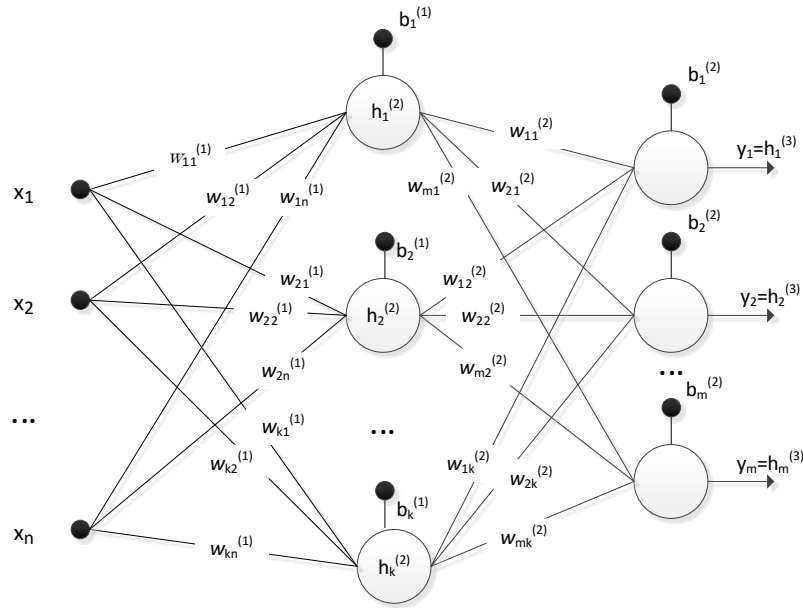
- A MLP is more complex than a single perceptron model as it is composed at least for 3 layers:
  - One input layer (pass-through neurons)
  - One or more hidden layers
  - One output layer
    - All the layers (except the output layer) include a bias neuron are **fully connected** to the next layer



Feed forward neural network (FNN)

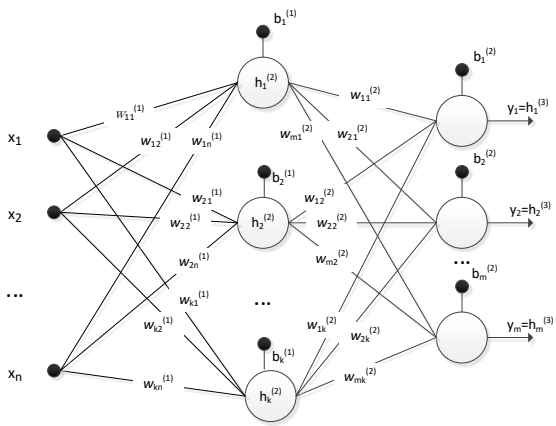
The signal flows only in one direction (from the inputs to the outputs)

Multilayer Perceptron (MLP)



Multilayer Perceptron (MLP)

- A MLP is composed of several **fully connected** layers:



Important!

By convention:

$$w_{ij}^{(l)}$$

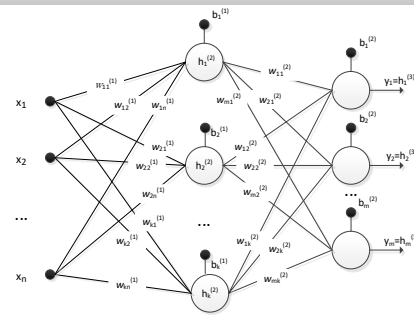
i refers to the node number of the connection in layer $l+1$

j refers to the node number of the connection in layer l

- In Feed Forward architecture neurons that belong to **one layer feed the next layer**
  - Communication is done layer by layer, starting from the input layer all the way through the hidden layers to the output layer.

Multilayer Perceptron (MLP)

- Notation



  - $w_{ij}^{(l)}$ (weights), $\boldsymbol{i}$ refers to the node number of the connection in layer $\boldsymbol{l+1}$ and $\boldsymbol{j}$ refers to the node number of the connection in layer $\boldsymbol{l}$
    - e.g. The weight of connection between **node 1 in layer 1** and **node 2 in layer 2** is denoted by $w_{21}^{(1)}$
  - $b_i^{(l)}$ (bias)**,** $\boldsymbol{i}$ is the node number in the layer $\boldsymbol{l+1}$
    - e.g. the **bias** in the second node in layer 2 (first hidden layer) is $b_2^{(1)}$
  - $h_j^{(l)}$ (node output), $\boldsymbol{j}$ denotes the node number in layer $\boldsymbol{l}$
    - e.g. the **output of node** 2 in layer 2 is denoted as $h_2^{(2)}$

Multilayer Perceptron (MLP)

- $a_i^{(l)}$ is the global input (sum of all inputs) into node i of layer l, including the bias term.
  - e.g. for the **1st node** in the **2nd layer**:

$$a_1^{(2)} = b_1^{(1)} + \sum_{j=1}^{n} w_{1j}^{(1)} x_j$$

Linear Combination

therefore $h_1^{(2)}$ can be defined as:

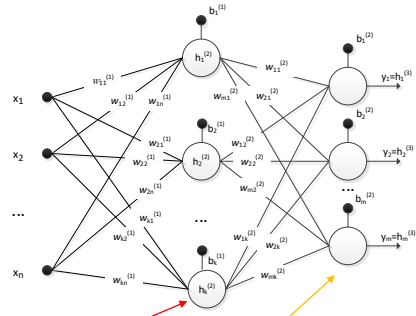
$$h_1^{(2)} = f(a_1^{(2)})$$

Activation (non linear model)

where f is the activation function.

Multilayer Perceptron (MLP)

- Based on this notation is possible to state that:



$$\mathbf{a}^{(2)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$
$$\mathbf{h}^{(2)} = f(\mathbf{a}^{(2)})$$

$$\mathbf{a}^{(3)} = \mathbf{W}^{(2)}\mathbf{h}^{(2)} + \mathbf{b}^{(2)}$$
$$\mathbf{y} = \mathbf{h}_{W,b}(\mathbf{x}) = \mathbf{h}^{(3)} = f(\mathbf{a}^{(3)})$$

$\mathbf{W}^{(1)}_{k\times n}$; $\mathbf{W}^{(2)}_{m\times k}$	weights matrices
$\mathbf{x} = [x_1 \dots x_n]$	instance
$\mathbf{b}^{(1)}$; $\mathbf{b}^{(2)}$	bias vectors applied to (1);(2)
$\mathbf{a}^{(l)}$	input of layer l
f	activation function
$\mathbf{h}^{(l)}$	output of layer l
$\mathbf{y} = \mathbf{h}^{(3)}$	desired output

Multilayer Perceptron (MLP)

- Based on this notation is possible to state that:

$$\mathbf{a}^{(2)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$
$$\mathbf{h}^{(2)} = f(\mathbf{a}^{(2)})$$

$$\mathbf{a}^{(3)} = \mathbf{W}^{(2)}\mathbf{h}^{(2)} + \mathbf{b}^{(2)}$$
$$\mathbf{y} = \mathbf{h}_{W,b}(x) = \mathbf{h}^{(3)} = f(\mathbf{a}^{(3)})$$

Generalizing for n layers

$$\mathbf{a}^{(l+1)} = \mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}$$
$$\mathbf{h}^{(l+1)} = f(\mathbf{a}^{(l+1)})$$

- The general feed forward process:
  - The output of layer $\boldsymbol{l}$ becomes the input to layer $\mathbf{l+1}$.
  - $\mathbf{h}^{(1)}$ refers to the input layer (input neurons; $\mathbf{h}^{(1)}$ is the output of the respective input "passthrough" neurons)
  - $\mathbf{h}^{(n_L)}$, (n_L is the number of layers) is the output of output layer.

Multilayer Perceptron (MLP)

- The behavior of neural networks depends on the two main aspects:
  - Architecture
    - How many layers?
    - How many neurons in each layer?
    - The weight of each connection
  - Activation Functions
    - There are several activation Functions:



Activation Functions

- Linear Function

Linear

  - The output is proportional to the input.

$$y = f(a) = ca$$

  - Main/important drawbacks:
    - All layers of the neural network collapse into one
      - If there are only linear activation functions, no matter how many layers in the neural network, the last layer will be a linear function of the first layer.
      - A neural network with exclusively a linear activation function is simply a linear regression model.
        - It has a very limited power and ability to handle complexity varying parameters of input data.

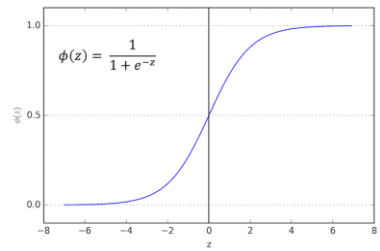
Activation Functions

- Non Linear

Non Linear

  - Essential for learning and modeling complex data
  - Address the problems of a linear activation function as they allow:
    - Backpropagation because they have a derivative function which is related to the inputs.
    - Multiple layers of neurons that are needed to learn complex data sets with high levels of accuracy.
  - Sigmoid/Logistic Function

$$y = f(a) = \frac{1}{1 + e^{-a}}$$



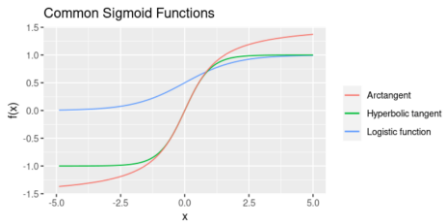
Activation Functions

- Non Linear

Non Linear

  - Hyperbolic Tangent Function

$$y = f(a) = \tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$$



Sigmoid function	Logistic function	tanh	arctan
Value in the limit $x \to -\infty$	0	-1	-π/2
Value at $x = 0$	0.5	0	0
Value in the limit $x \to \infty$	1	1	π/2
Converges	Fast	Very fast	Very slow

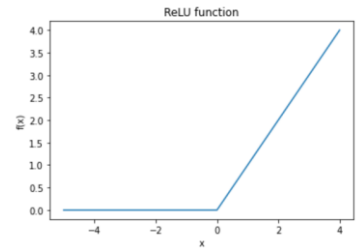
https://deepai.org/machine-learning-glossary-and-terms/sigmoid-function

Activation Functions

Non Linear

- Non Linear
  - ReLU (Rectified Linear Unit)

$$f(a) = \max(0, a)$$



  - The ReLU function presents some advantages:
    - It is very fast to calculate.
    - An activation potential in a biological neural network does not continue to change for negative inputs, so the ReLU seems closer to the biological reality if a goal is to mimic biological systems.
    - for $x > 0$ has a constant gradient of 1
    - However, the zero gradient for $x < 0$ can create a problem (the zero gradient problem)
    - There are some variants of ReLU: Leaky ReLU, Parametric ReLU

Activation Functions

Non Linear

- Non Linear
  - Softmax
    - Last layer of a feed forward neural network
    - Turns a vector of p real values into a vector of p real values that sum to 1.

$$y = f(\mathbf{a}) = \frac{e^{a_i}}{\sum_{j=1}^{p} e^{a_j}} \quad ; \quad \mathbf{a} = [a_1, \ldots, a_p]$$

    - The input values a_i can be positive, negative, zero:
      - Softmax outputs range between 0 and 1
      - The output of Softmax can be interpreted as probabilities.
        - If one of the inputs is small or negative
          - Softmax turns it into a small probability
        - If an input is high then
          - It turns it into a large probability

    but it will always remain between 0 and 1.

Activation Functions

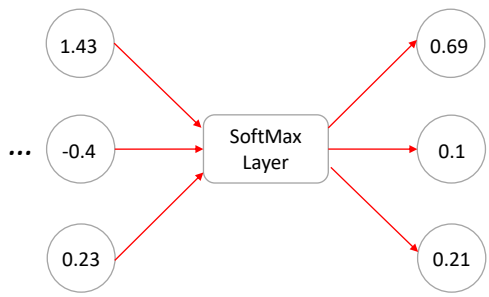
- Non Linear
  - Softmax

Non Linear

$$y = f(\mathbf{a}) = \frac{e^{a_i}}{\sum_{j=1}^{p} e^{a_j}} \quad ; \quad \mathbf{a} = [a_1, \dots, a_p]$$

- Able to handle multiple classes
  - Can receive the inputs from other activation functions, normalize the outputs, and give the probability of the input belongs to a specific class
  - Applied in the output layer (soft max layer) when the neural network intends to perform a **multiclass classification**

Activation Functions



Output layer

$$\begin{bmatrix} 1.43 \\ -0.4 \\ 0.23 \end{bmatrix}$$

$$y = \frac{e^{a_i}}{\sum_{j=1}^{p} e^{a_j}}$$

Softmax activation function

Probabilities

$$\begin{bmatrix} 0.69 \\ 0.1 \\ 0.21 \end{bmatrix}$$

Non Linear

1.Input.
The softmax function takes a vector of real numbers (outputs from the final layer of the neural network).

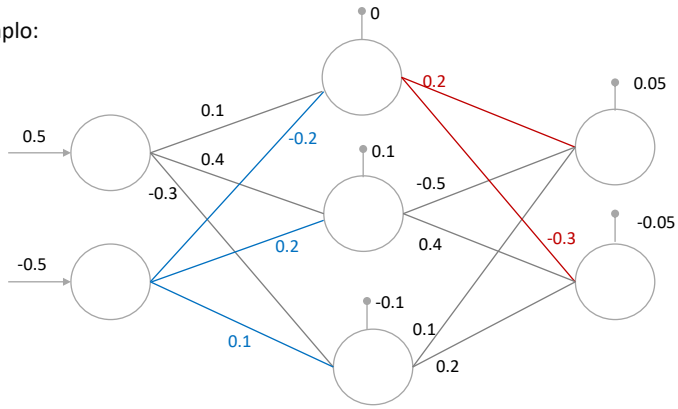
2. Exponentiation.
Ensures all values become positive.

3. Normalization.
Guarantees the output values sum to 1, the output values lie between 0 and 1 (probabilities)

4. Interpretability.
Softmax transforms the raw outputs into probabilities, making the network's predictions easier to understand and analyze.

Example

- Exemplo:



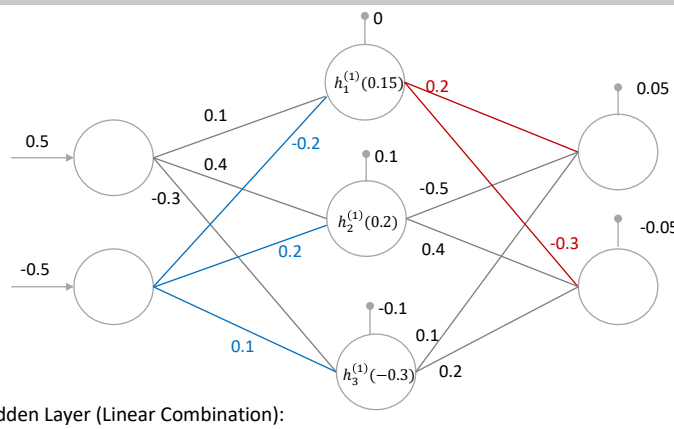
- Initial values (Random):

$$x = \begin{bmatrix} 0.50 \\ -0.50 \end{bmatrix}$$

$$W^{(1)} = \begin{bmatrix} 0.1 & -0.2 \\ 0.4 & 0.2 \\ -0.3 & 0.10 \end{bmatrix} \quad b^{(1)} = \begin{bmatrix} 0.00 \\ 0.10 \\ -0.10 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 0.20 & -0.50 & 0.10 \\ -0.30 & 0.40 & 0.20 \end{bmatrix} \quad b^{(2)} = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

Example

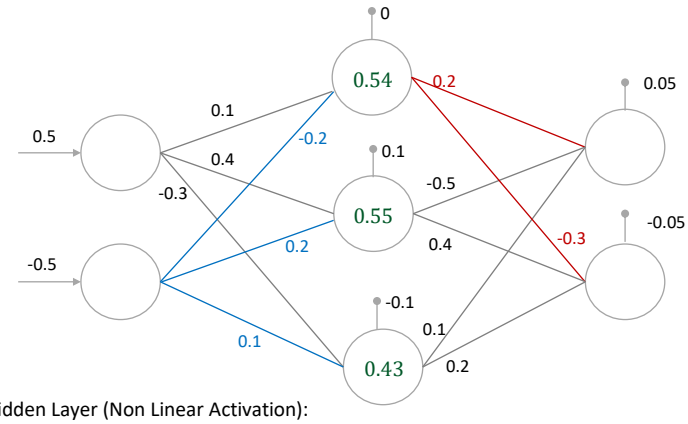


- Hidden Layer (Linear Combination):

$$\mathbf{a}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

- neuron 1: $a_1 = 0.1(0.5) + (-0.2)(-0.50) + 0 = 0.15$
- neuron 2: $a_2 = 0.4(0.5) + (0.2)(-0.50) + 0.1 = 0.20$ => $\mathbf{a}^{(1)} = \begin{bmatrix} 0.15 \\ 0.20 \\ -0.30 \end{bmatrix}$
- neuron 3: $a_3 = -0.3(0.5) + 0.1(-0.50) - 0.1 = -0.3$

Example

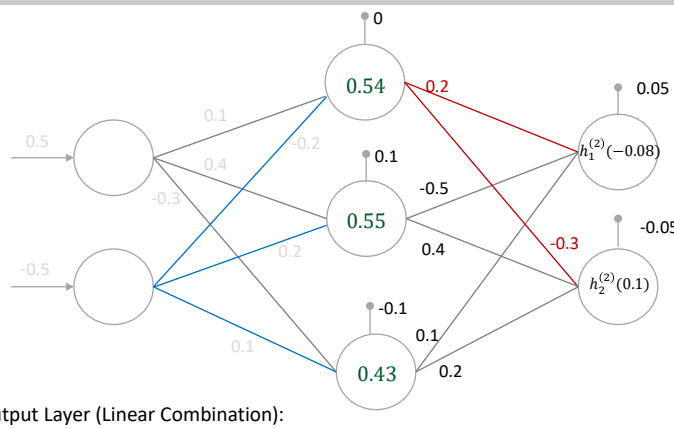


- Hidden Layer (Non Linear Activation):
  - Sigmoid/logistic function

$$\boldsymbol{h}^{(1)} = \sigma\left(\boldsymbol{a}^{(1)}\right) = \frac{1}{1+e^{-a}}$$

$$h_1 = \sigma(0.15) = 0.54;\ h_2 = \sigma(0.2) = 0.55; h_3 = \sigma(-0.30) = 0.43 \quad \Rightarrow \quad h^{(1)} = \begin{bmatrix} 0.54 \\ 0.55 \\ 0.43 \end{bmatrix}$$

Example

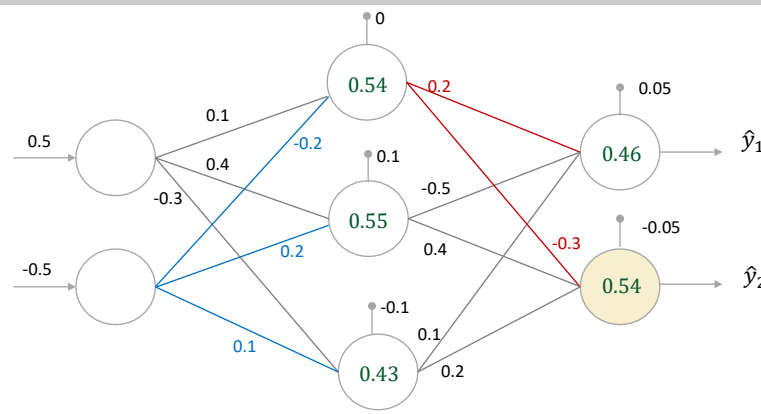


- Output Layer (Linear Combination):

$$\mathbf{a}^{(2)} = \mathbf{W}^{(2)}\mathbf{x} + \mathbf{b}^{(2)}$$

- neuron 1: $a_1 = 0.2(0.54) + (-0.5)(0.55) + 0.1(0.43) + 0.05 = -0.08$
- neuron 2: $a_2 = -0.3(0.54) + (0.4)(0.55) + 0.2(0.43) - 0.05 = 0.1$ => $\mathbf{a}^{(2)} = \begin{bmatrix} -0.08 \\ 0.1 \end{bmatrix}$

Example



- Output Layer (Non Linear Activation):
  - Softmax

$$p_i = \frac{e^{a_i}}{e^{a_1} + e^{a_2}}$$

$$p_1 = \frac{0.92}{0.92+1.10} = 0.46;\ p_2 = \frac{1.10}{0.92+1.10} = 0.54 \quad \Rightarrow \quad \hat{y} = [0.46, 0.54]$$

MLP

- Learning/Training
  - Determine the weights $w_{ij}^{(l)}$ ($\mathbf{W}^{(l)}$) and bias $b_i^{(l)}$ ($\mathbf{b}^{(l)}$)
    - Find out how each connection weight and each bias term should be considered in order to reduce the error.
  - A cost function has to be minimized:

$$J(\mathbf{W}, \boldsymbol{b}) = \frac{1}{N}\sum_{z=1}^{N}(y_z - \hat{y}_z)^2$$

where for each instance $\boldsymbol{z}$, $\hat{y}_z$ is the predicted output and y_z the true value .

  - The optimization, calculation of the new $w_{i,j}^{(l)}$ and $b_i^{(l)}$ should be performed through gradient descent:

$$w_{i,j}^{(l)} = w_{i,j}^{(l)} - \alpha \frac{\partial}{\partial w_{ij}^{(l)}} J(\mathbf{W}, \boldsymbol{b})$$

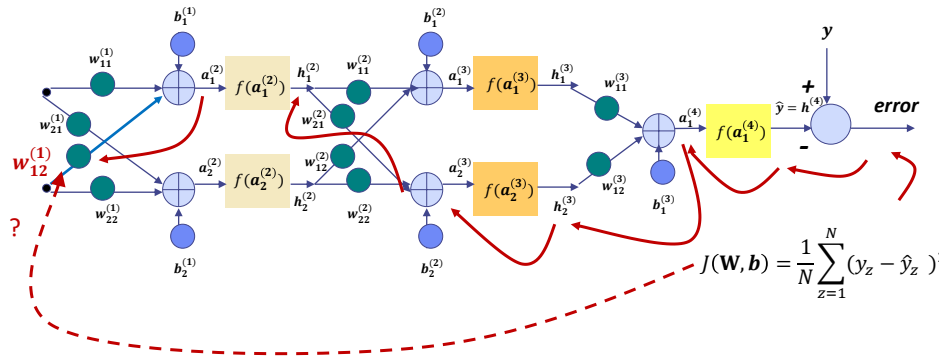
$$b_i^{(l)} = b_i^{(l)} - \alpha \frac{\partial}{\partial b_i^{(l)}} J(\mathbf{W}, \boldsymbol{b})$$

The gradient determination is complex due to the hidden layers.

Solution? **Back propagation!**

MLP

- Learning: Back Propagation Algorithm



How to compute ? $\frac{\partial J}{\partial w_{12}^{(1)}}$

MLP

- Back Propagation Algorithm

1. Start from a set of training instances (batch/mini batch) and it goes through the entire training instances multiple times (epochs).
2. Each batch/mini batch is passed to the input layer, then it is computed through all the layers (hidden) until it reaches the output of output layer (**forward pass**)
3. **The error is computed (cost function)**
4. The contribution for the error of each output connection is computed. This determination is done analytically through the **chain rule** until the input layer is reached (**backward pass**)
5. This back propagation pass efficiently measures the error gradient across all connection weights in the network by propagating the error gradient backward.
6. Finally the algorithm performs the calculation of new parameters (gradient descent) based on the gradient that was computed.

MLP

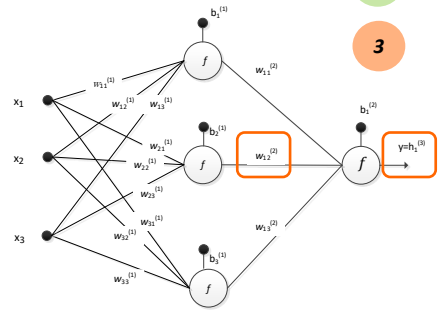
- Back Propagation Algorithm

$$\hat{y} = h_{W,b}(x) = h_1^{(3)} = f(w_{11}^{(2)} h_1^{(2)} + w_{12}^{(2)} h_2^{(2)} + w_{13}^{(2)} h_3^{(2)} + b_1^{(2)})$$

$$h_1^{(3)} = f(a_1^{(3)})$$

$$a_1^{(3)} = w_{11}^{(2)} h_1^{(2)} + w_{12}^{(2)} h_2^{(2)} + w_{13}^{(2)} h_3^{(2)} + b_1^{(2)}$$

1 2 3



$$J(\mathbf{W}, \boldsymbol{b}) = \frac{1}{N}\sum_{z=1}^{N}(y_z - \hat{y}_z)^2 = \frac{1}{N}\sum_{z=1}^{N}(y_z - h_1^{(3)})^2$$

$$\frac{\partial J}{\partial w_{12}^{(2)}} \quad ?$$

- Solution based on the "Chain Rule"

"If a variable z depends on the variable y, which itself depends on the variable x (i.e., y and z are dependent variables), then z, via the intermediate variable of y, depends on x as well."

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

MLP

- Back Propagation Algorithm

4

$$J = \frac{1}{N}\sum_{z=1}^{N}(y_z - h_1^{(3)})^2$$

$$h_1^{(3)} = f(a_1^{(3)})$$

$$a_1^{(3)} = w_{11}^{(2)} h_1^{(2)} + w_{12}^{(2)} h_2^{(2)} + w_{13}^{(2)} h_3^{(2)} + b_1^{(2)}$$

$$\frac{\partial J}{\partial w_{12}^{(2)}} \quad ?$$

- Chain rule

$$\frac{\partial J}{\partial w_{12}^{(2)}} = \frac{\partial J}{\partial h_1^{(3)}}\frac{\partial h_1^{(3)}}{\partial a_1^{(3)}}\frac{\partial a_1^{(3)}}{\partial w_{12}^{(2)}}$$

The same reasoning can be applied for the weights in this layer that directly relates with the output.

$$\frac{\partial J}{\partial h_1^{(3)}} = -(y - h_1^{(3)})$$

$$\frac{\partial h}{\partial a} = f'(a)$$
derivative of the activation function

$$\frac{\partial a_1^{(3)}}{\partial w_{12}^{(2)}} = h_2^{(2)}$$

For the **weights connecting the output layer** is possible to state:

$$\delta_i^{(n_L)} = -(y_i - h_i^{(n_L)}) f'(a_i^{(n_L)})$$ Local Gradient

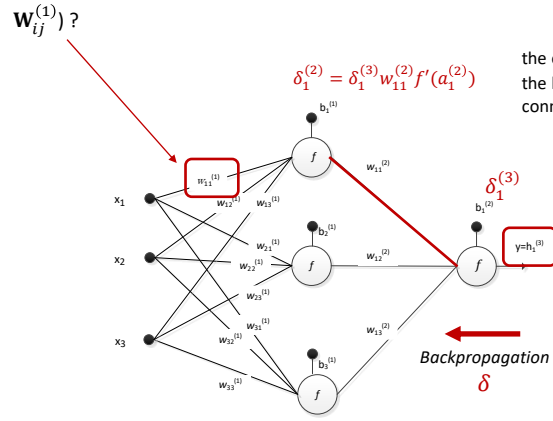
$$\frac{\partial J}{\partial \mathbf{W}_{ij}^{(l)}} = \delta_i^{(l+1)} h_j^{(l)}$$

where, i is the node number and l refers to the layer

https://adventuresinmachinelearning.com/neural-networks-tutorial/

MLP

- Back Propagation Algorithm
  - Weights that feed the other layers that **are not directly connected to the cost function** (e.g. $\mathbf{W}_{ij}^{(1)}$) ?



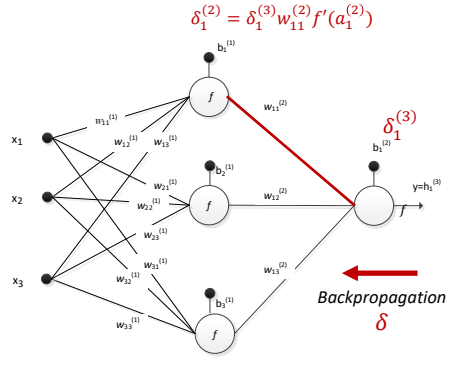
the output layer $\delta_1^{(3)}$ is communicated to the hidden node by the **weight** of the connection

MLP

- In general form is:

$$\delta_j^{(l)} = \delta_1^{(l+1)} w_{1j}^{(l)} f'(a_j^{(l)})$$

5



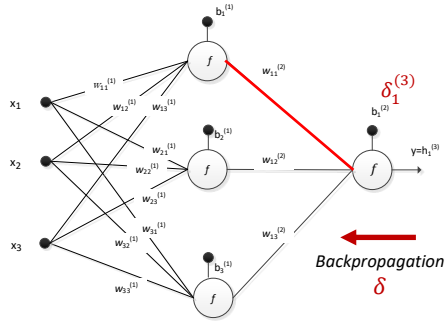
the output layer $\delta_1^{(3)}$ is communicated to the hidden node **by the weight** of the connection

$$\delta_i^{(n_L)} = -(y_i - h_i^{(n_L)}) f'(a_i^{(n_L)})$$

MLP

- $\mathbf{W}_{ij}^{(1)}$?

$\delta_1^{(2)} = \delta_1^{(3)} w_{11}^{(2)} f'(a_1^{(2)})$



6

$$\frac{\partial J}{\partial \mathbf{W}_{ij}^{(l)}} = h_j^{(l)} \delta_i^{(l+1)}$$

$$\frac{\partial J}{\partial \boldsymbol{b}_i^{(l)}} = \delta_i^{(l+1)}$$

These two expressions allow to calculate the variation in $\boldsymbol{J}$ due to the changes of all the parameters in the MLP $(\mathbf{W}_{ij}^{(l)}, \mathbf{b}_i^{(l)})$

Learning Process
update weights
update bias

$$w_{i,j}^{(l)} = w_{i,j}^{(l)} - \alpha \frac{\partial}{\partial w_{ij}^{(l)}} J(\mathbf{W}, \boldsymbol{b})$$

$$b_i^{(l)} = b_i^{(l)} - \alpha \frac{\partial}{\partial b_i^{(l)}} J(\mathbf{W}, \boldsymbol{b})$$

α is learning rate

MLP

- Some second order optimization algorithms might also be applied
  - Newton Algorithm
  - Levenberg Marquardt
  - …

MLP

- Regression MLP
  - MLP can be applied for regression problems
    - If the objective is to predict:
      - a single value the output layer must have 1 neuron. Its output is the desired prediction.
      - multiple values, the output layer must have several neurons, 1 per output dimension.
    - When the objective is regression, usually there is a linear function in the output neurons, which means that they are free to output any range of values.
      - But some activation functions might be applied according to the specificities of the problem:
        - ReLU or $softplus(z) = \log(1 + \exp(z))$ to guarantee that the output will always be positive
        - Logistic Function or hyperbolic tangent to guarantee that the output will be within a given range of values (scaled from 0 to 1 or -1 to 1)
      - The loss function is typically the MSE

MLP

- Regression MLP
  - Typical Architecture (ANN tuning)

Hyperparameter	Typical Value
#input neurons	One per input feature
#hidden layers	Depends on the problem, typically 1 to 5
#neurons per hidden layer	Depends on the problem, typically 10 to 100
#output neurons	1 per predicted dimension
Hidden activation	ReLU
Output activation	Linear ReLU/softPlus for positive ouptuts logistics/tanh for bounded outputs
Loss Function	MSE or MAE

Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, ISBN-13: 978-1492032649, O'Reilly 2022

MLP

- Classification MLP
  - MLP's are applied for several types of classification problems
    - Binary Classification ($labels = 1; classes = 2$):
      - The output layer has 1 neuron with logistic activation function. The output is a number between 0 and 1 that corresponds to the predicted probability of the positive class.
    - MultiLabel Classification ($labels \geq 2; classes = 2$):
      - The output layer has the same number of neurons that the number of labels.
      - The activation function should be the logistic function.
    - MultiClass Classification ($labels = 1; classes \geq 2$)
      - One output neuron per class
      - The softmax activation function should be applied to the whole output layer. The softmax assures that all the estimated probabilities are between 0 and 1, their sum should be 1 and the classes are exclusive.
    - The loss function is typically the cross entropy

MLP

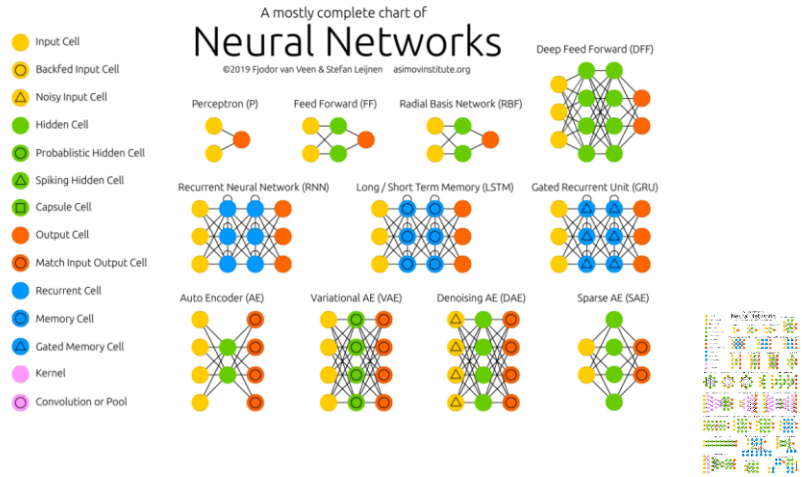
- Classification MLP
  - Typical Architecture (ANN tuning)

Hyperparameter	Binary Class.	Multilabel Class.	Multiclass Class.
#input neurons	One per input feature	One per input feature	One per input feature
#hidden layers	Depends on the problem, typically 1 to 5	Depends on the problem, typically 1 to 5	Depends on the problem, typically 1 to 5
#neurons per hidden layer	Depends on the problem, typically 10 to 100	Depends on the problem, typically 10 to 100	Depends on the problem, typically 10 to 100
#output neurons	1	1 per label	1 per class
Hidden activation	ReLU	ReLU	ReLU
Output activation	Logistic	Logistic	Softmax
Loss Function	Cross Entropy	Cross Entropy	Cross Entropy

gray: the same as Regression

ANN

- There are several types of Neural Networks:



https://www.asimovinstitute.org/neural-network-zoo/